Chapter 4: QL Language Fundamentals

Rod Trent

Chapter 4: QL Language Fundamentals

Download this chapter as a PDF for offline reading:
Chapter-04-QL-Language-Fundamentals.pdf

QL is the declarative, object-oriented query language that powers CodeQL. This chapter introduces the core concepts you need to read and write queries: the declarative model, predicates, formulas, classes, modules, recursion, and aggregates.

Declarative Model

QL is based on Datalog. You describe *what* you want, not *how* to compute it. A query is a logical formula that the engine evaluates against the relational database.

The basic shape of a query is:

```
from /* variable declarations */
where /* logical formula */
select /* results */
```

You declare typed variables, constrain them with a formula, and select the values that satisfy the formula. The engine finds all solutions; you do not write loops or iteration.

Example (Pythagorean triples with small integers):

```
from int x, int y, int z
where x in [1..10] and y in [1..10] and z in [1..10] and
      x*x + y*y = z*z
select x, y, z
```

Because everything is a logical relation, the same model scales from simple filters to interprocedural data-flow analysis.

Predicates, Formulas, Classes, and Modules

Predicates

A predicate defines a relation. It can be a non-member predicate, a member predicate, or a characteristic predicate.

```
predicate isSmall(int i) {
  i in [1..10]
}

int successor(int i) {
  result = i + 1 and i in [1..9]
}
```

Member predicates belong to a class and can use `this`. Characteristic predicates restrict the values that belong to a class.

Formulas

Formulas use the usual logical connectives (`and`, `or`, `not`) and quantifiers (`exists`, `forall`). A formula holds or does not hold for a given assignment of values.

Classes

A class in QL does not create objects; it represents a set of values that satisfy a logical property.

```
class SmallInt extends int {
  SmallInt() { this in [1..10] }

  int square() { result = this * this }
}
```

You can then write:

```
from SmallInt x, SmallInt y, SmallInt z
where x.square() + y.square() = z.square()
select x, y, z
```

Modules

Modules group related predicates, classes, and other modules. They support reuse and avoid name clashes. You import a module with an `import` statement.

```
import javascript

from Function f
select f.getName()
```

Recursion and Aggregates

Recursion

QL supports recursion. A recursive predicate is evaluated to its least fixed point: the engine starts with the empty set and repeatedly applies the predicate until the set stops growing.

```
int getANumber() {
  result = 0
```

```
  or
  result <= 100 and result = getANumber() + 1
}
```

Transitive closure is a common pattern. QL provides the `+` operator as shorthand:

```
// getAParent+ means one or more applications of getAParent
Person getAnAncestor() { result = this.getAParent+() }
```

Aggregates

Aggregates summarize sets of values. Common ones include `count`, `sum`, `min`, `max`, and `avg`.

```
from Function f
where count(f.getACall()) > 10
select f, count(f.getACall())
```

Aggregates are evaluated after the rest of the formula, so you can use them to filter or to project summary values.

Putting It Together

A minimal query that uses several of these ideas lives in the companion repository at `examples/chapter04/small-int-triples.ql`:

```
/**
 * @name Small integer Pythagorean triples
 * @description Finds Pythagorean triples using small integers.
 * @kind problem
 * @id mustlearn/small-int-triples
 */
import javascript

class SmallInt extends int {
  SmallInt() { this in [1..20] }
  int square() { result = this * this }
}

from SmallInt x, SmallInt y, SmallInt z
where x.square() + y.square() = z.square() and x <= y
select x, y, z, "Triple: " + x + ", " + y + ", " + z
```

(The `import javascript` line is only needed if you want to run the query against a JavaScript database; the rest is pure QL.)

Next Steps

You now have the basic vocabulary of QL: declarative queries, predicates, classes, modules, recursion, and aggregates. In the next chapter we look at writing custom predicates that you can reuse across queries.

All examples referenced above live in the companion repository: github.com/rod-trent/MustLearnCodeQL under `examples/chapter04/`.

Must Learn CodeQL — Rod Trent
Companion repository: github.com/rod-trent/MustLearnCodeQL